



Final Technical Report

PeekIR Members:

Havish Bandapalli

Sean Conway

Trisha Komitla

Vidhya Kommineni

Rayaan Naveed

Rajat Sarkar

Sponsor: Dr. Logan Porter

Technical Advisor: Dr. Muhammad Faeyz Karim

Revision Date: 4/27/2026

Revision Number: 1

Table of Contents

Introduction.....	2
Project Value Proposition.....	2
Stakeholders and Intended Users.....	3
PeekIR Team Members.....	3
Project Overview.....	4
Executive Summary.....	4
Problem Statement.....	5
Project Scope & Objectives.....	5
Component Selection.....	6
Functional Specifications.....	14
System Design Overview.....	16
Testing and Validation Approach.....	17
Electronic/Hardware Designs.....	28
Gantry Hardware System.....	28
Metal Detector Wand.....	29
Software Design.....	30
Software Architecture Overview.....	30
Data Acquisition & Processing.....	31
Mechanical Design.....	34
Mechanical System Overview.....	34
Motion & Positional System.....	35
Structural Design & Packaging.....	37
Overall Theory of Operation for Mechanical System.....	39
Project Status.....	40
Cost Analysis.....	40
Bill of Materials.....	41
Reproduction Cost.....	43
Product Evaluation.....	43

Introduction

This report presents the final technical design and development status of the multi-sensor concealed threat detection system developed by Team PeekIR as part of the ESET Capstone Program. The project was undertaken to develop a proof-of-concept prototype for a practical, intelligent, and cost-conscious security screening solution capable of assisting with concealed threat detection in real world environments.

The system developed by Team PeekIR was designed as a prototype platform that combines multiple sensing methods into one integrated solution. The prototype incorporates millimeter Wave Synthetic Aperture Radar (mmWave SAR), infrared imaging (IR), visible light imaging, and metal detection, with Machine Learning (ML) models for suspicious object detection in IR imaging and facial recognition in visible light imaging into an integrated dashboard for optimal ease of use and security rigor. Unlike traditional single-sensor approaches, this system is intended to improve screening capability by combining multiple sensor modalities and intelligent detection models into a more informed threat evaluation platform.

This document serves as the formal final technical report for the project. It summarizes the design process, subsystem development, major engineering decisions, testing methods, and final project status. It also documents how the team approached the electrical, software, and mechanical challenges involved in creating an integrated prototype system.

Project Value Proposition

PeekIR seeks to offer a superior security solution for travel, event venues, schools, and other security sensitive environments by offering an integrated platform offering insights across a range of sensing capabilities to offer a more comprehensive safety system than any currently on the market while minimizing manpower and cost requirements. Current systems offer only single modality sensing such as TSA full body scanners, traditional walk-through metal detectors, and facial recognition systems. PeekIR's platform, conversely, offers a fully integrated hardware and software system for comprehensive security evaluation. TSA full body scanners, which also use mmWave technology but highlight potential threats with a yellow box rather than detailed images of detected objects. PeekIR's mmWave system offers a superior solution to this by

providing detailed 3D imaging of concealed objects in addition to the multi sensor integration and intelligent detection features.

Stakeholders and Intended Users

The project is sponsored by the Electronic Systems Engineering Technology(ESET) department at Texas A&M University headed by Dr. Logan Porter. The project's technical advisor is Dr. Muhammad Faeyz Karim.

The intended users for the PeekIR system are organizations overseeing security critical locations and events where concealed weapon and personnel detection solutions are required for public and operational safety. Such organizations could include government entities such as the Transportation Security Administration(TSA), educational institutions, and private companies responsible for event and retail security.

PeekIR Team Members

The team consists of Havish Bandapalli (Test Engineer), Sean Conway (Systems Integration Engineer), Trisha Kommitla (Hardware Engineer), Vidhya Kommineni (Software Engineer), and Rayaan Naveed (Software Engineer), and Rajat Sarkar (Project Manager/Hardware Engineer).

Project Overview

Executive Summary

The goal of this project is to develop and improve a functional prototype of a concealed weapon detection system. The prototype is developed for public environments where there is concentrated foot traffic such as campuses, airports, venues, offices and more. The core objective of PeekIR's iteration on the gantry system are improving detection accuracy, data acquisition quality, as well as integration of sensing modalities through a centralized user-interface which interacts with the system.

This project is an iteration upon a previous ESET capstone project which provided the PeekIR team with mmWave radar and gantry hardware and a partially functional mmWave imaging software workflow. PeekIR set out to develop a fully integrated multimodal security system. Goals included reworking the mmWave radar imaging to significantly improve image quality and scan efficiency, integrating facial detection and recognition against a blacklisted persons database, integrating infrared imaging with object detection, and integrating metal detection all into a unified user-friendly dashboard.

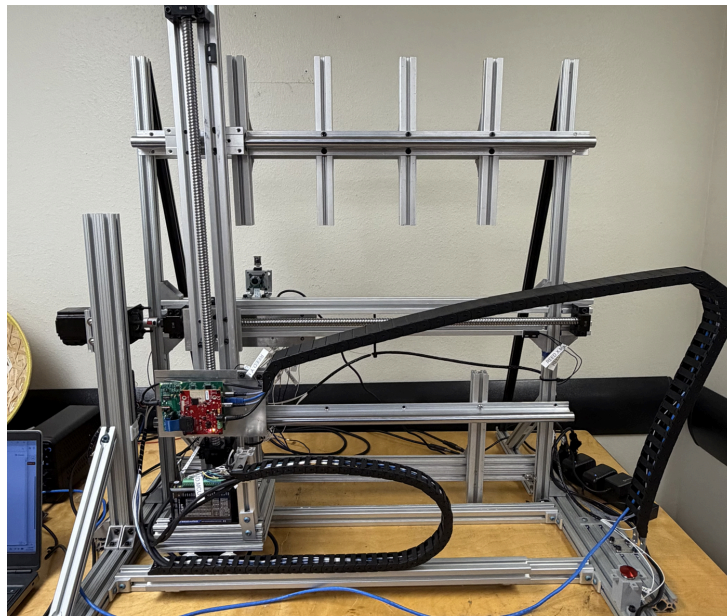


Figure 1: PeekIR Gantry Hardware System

Problem Statement

Current systems lack the accuracy and intelligence needed for reliable real world threat detection. Currently available weapon detection systems rely on single modality sensing, which frequently results in false positives, blind spots, and lack of context. Reliance of current systems on manual inspection also leaves them prone to human error.

Project Scope & Objectives

The PeekIR team has developed a functional prototype of a concealed weapon and threat detection system integrating mmWave Radar, Infrared, and visible light imaging with metal detection and Machine Learning(ML).

The mmWave SAR subsystem is used to screen individuals for weapons concealed under clothing. Scan capture controls are integrated into the dashboard along with an embedded 3D interactive viewer for observing completed scans.

The infrared imaging subsystem provides a live infrared feed to the dashboard and applies ML based object recognition to detect weapons to captured infrared images.

The visible light imaging subsystem provides a live visible light camera feed to the dashboard and applies ML to detect faces in captured images and check identity against a list of blacklisted or wanted individuals.

The metal detection subsystem consists of a rechargeable wireless metal detector security wand which integrates with the dashboard over Bluetooth to surface alerts in addition transmitting battery status to and indicating metal detection through on board indicator LEDs.

Limitations of the project scope include that PeekIR set out to develop an exploratory proof of concept prototype for an integrated multimodal security system, not a production ready system. The team set out to develop a manned system which leverages ML based insights to enhance security personnel, not to fully replace them. The mmWave subsystem is limited in its scanning speed, area, and depth, to performing a 150mm x 150mm scan imaging at a depth of 200mm -

400mm in approximately 30 minutes. The SAR visualization workflow does not include automatic weapon detection and surfaces a 3D interactive viewer which is for manual inspection only. As the facial recognition and infrared object detection features rely on machine learning models, a risk of false positive and false negative results cannot be eliminated entirely, and instead the team optimized to mitigate the risk of incorrect inference. The scope is also limited by the acknowledgement that as an iterative project, the gantry hardware system and parts of the mmWave SAR software workflow were inherited from previous capstone teams.

Component Selection

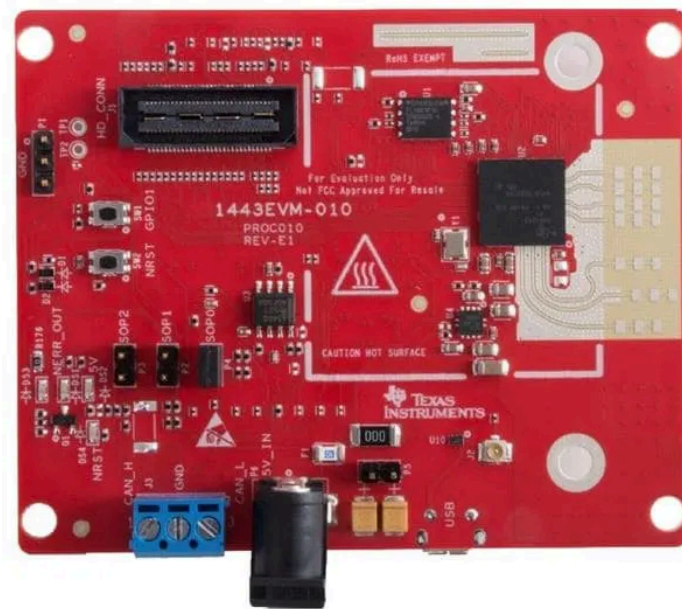


Figure 2: IWR1443BOOST Evaluation Board

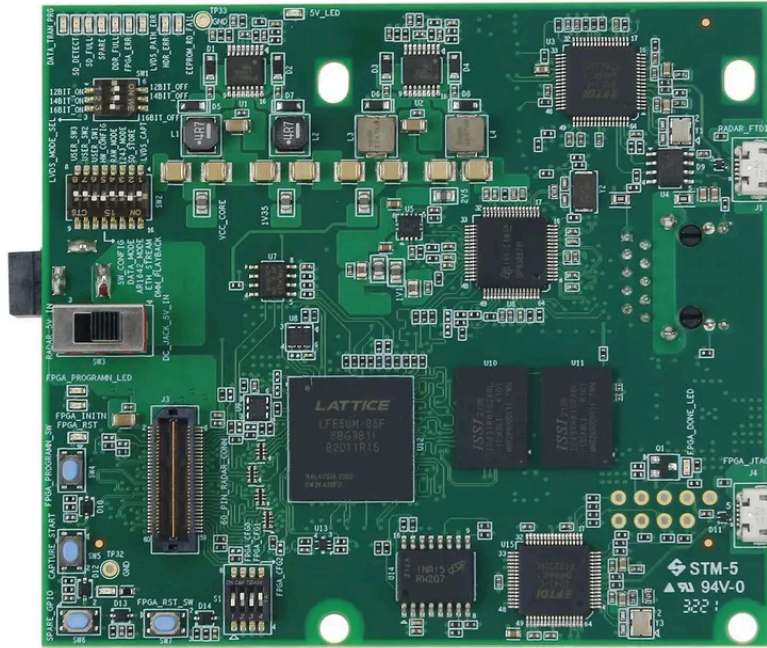


Figure 3: DCA1000EVM Evaluation Board

The central subsystem of the PeekIR platform is the mmWave SAR 3D imaging subsystem. To deliver this functionality, the Texas Instruments(TI) IWR1443BOOST evaluation board and DCA100EVM evaluation board were selected. As these components were inherited from previous capstone projects working on mmWave SAR applications, the PeekIR team did not select to purchase these components. The combination of these two evaluation boards, however, was a suitable choice for PeekIR's application because they offer a developer friendly platform to collect mmWave Radar data and interface with TI's mmWave Studio application and mmWave Software Development Kit(SDK). The IWR1443BOOST offers a single chip 76 GHz to 81 GHz mmWave sensor evaluation module suitable for detecting concealed objects safe non-ionizing radiation with a configurable Application Program Interface (API) well suited for programmatically configuring radar configuration. The DCA1000EVM provides an evaluation module for real time data capture and streaming, enabling the PeekIR to receive data collected by the IWR1443BOOST as raw binary files, enabling custom visualization and reconstruction workflows.

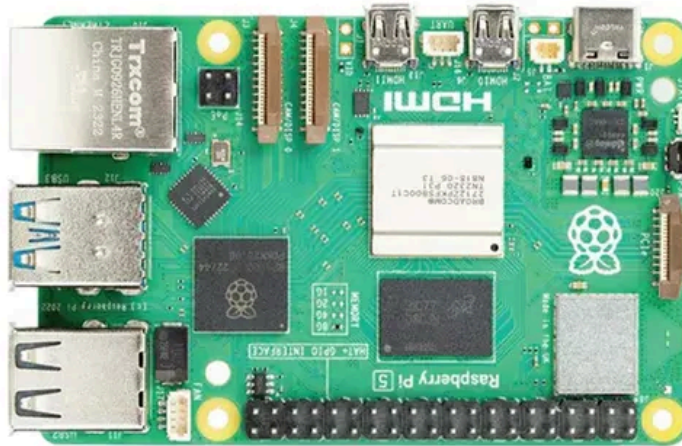


Figure 4: Raspberry Pi 5

For managing gantry positional control, visible light camera streaming, and Infrared camera streaming, the team selected a Raspberry Pi 5 for its highly favorable combination of low cost, low form factor, and developer friendly, versatile functionality. The combination of features that made the Raspberry Pi 5 optimal for this system include its WiFi connectivity, availability of GPIO pins with hardware pulse width modulation(PWM) necessary for driving stepper motors, scriptability with python, and hardware interfaces for Camera Serial Interface(CSI) ports (for interfacing with visible light camera), and USB ports (for interfacing with Infrared Camera). While other controller solutions may offer a subset of these capabilities and hardware interfaces, none offer as favorable of a development experience suited for the PeekIR system as the Raspberry Pi 5.

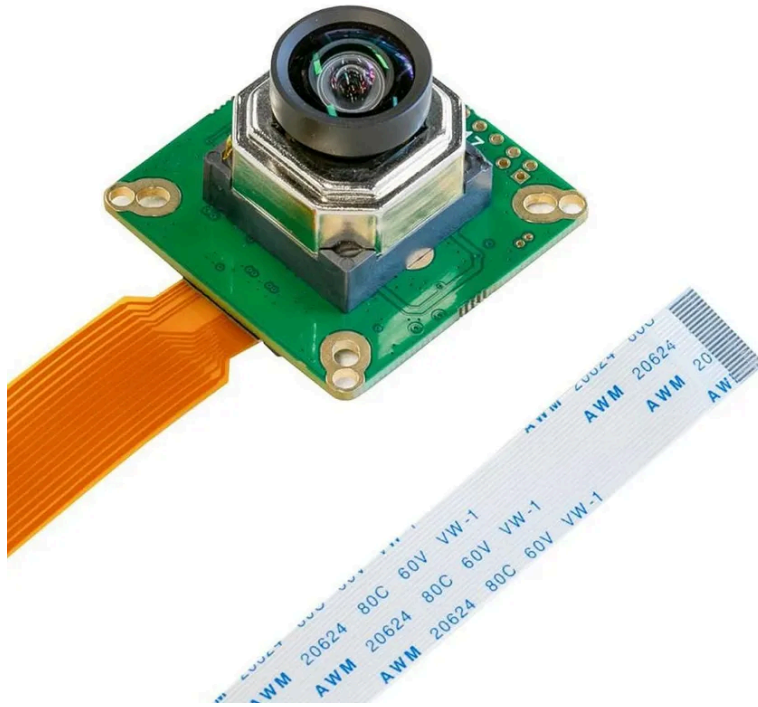


Figure 5: Arducam Pi Camera 3 Wide

For the visible light imaging camera, Arducam Pi Camera 3 Wide was selected for its ability to interface easily with the Raspberry Pi 5, suitable field of view and resolution for facial recognition applications and cost effectiveness as it was inherited from previous capstone teams.



Figure 6: FLIR Lepton UWFOV Infrared Camera



Figure 7: PureThermal Mini USB IR Camera Breakout Board

For the infrared imaging subsystem, initially a TOPDON TC001 infrared camera was considered, an infrared camera module designed for interfacing with Android smart phones with community developed libraries for Raspberry Pi integration support. Upon consultation with Dr. Porter, however, the determination was made to purchase a FLIR Lepton UWFOV infrared camera and breakout board instead in order to mitigate the risk of depreciated libraries blocking integration efforts.



Figure 8: CL57T Closed Loop Stepper Driver

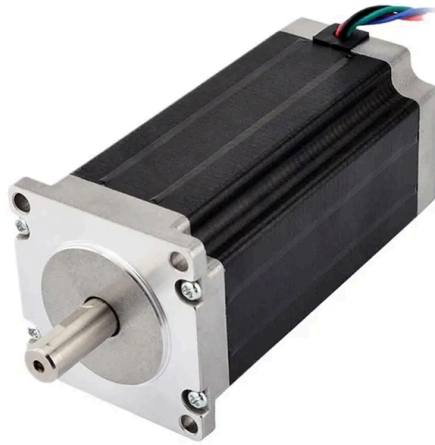


Figure 9: 23HS45-4304D Stepper Motor

The PeekIR team opted to replace the existing stepper drivers(TB6600) and stepper motors(23HS45-4204S) with new stepper drivers(CL57T) and stepper motors(23HS45-4304D) which feature closed loop encoder feedback for enhanced accuracy in gantry positional control. While initial plans involved reading the encoder feedback externally using a differential line receiver and microcontroller module to count encoder pulses and enhance positional estimation, further testing revealed that method to be less accurate than pure software based positional estimation. The change to closed loop encoder stepper drivers and motors however, still constitutes a substantial improvement in positional accuracy, because the new closed loop encoder hardware improves system robustness by reducing the occurrence to missed or added steps of the motor as it transplanted from Pi generated step pulses to physical motion.

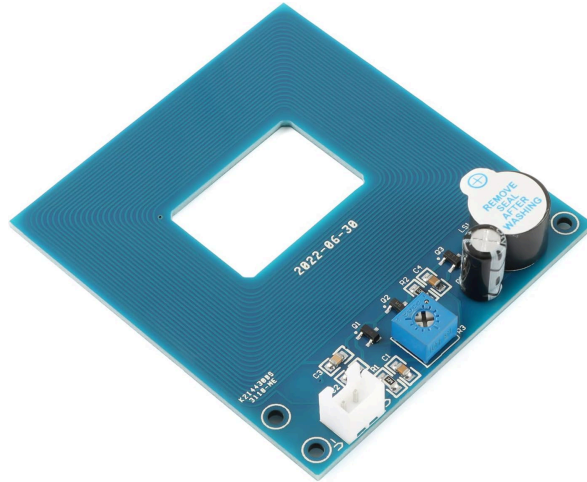


Figure 10: JESSINIE Metal Detector Module

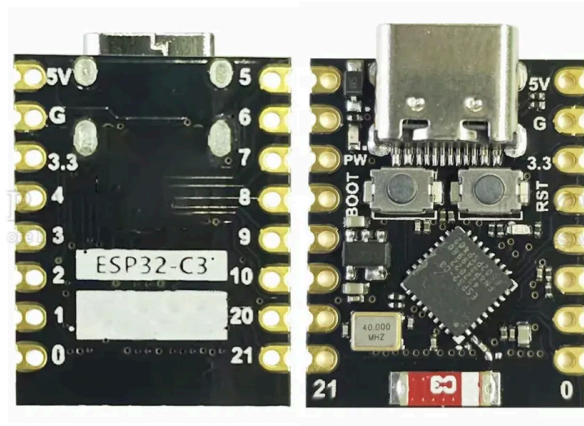


Figure 11: ESP32C3 Supermini Development Board

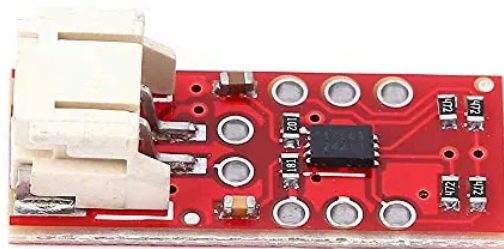


Figure 12: MAX17043 LiPo Fuel Gauge Module

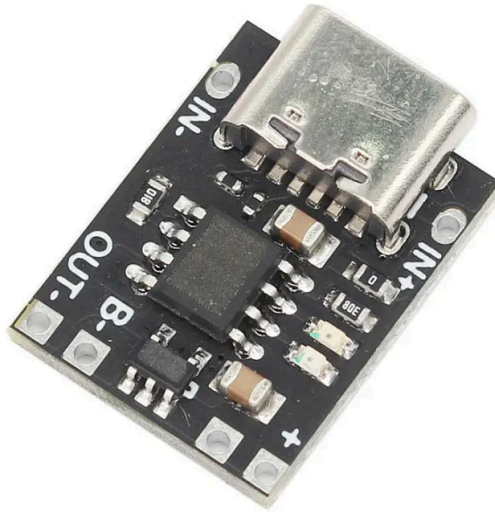


Figure 13: TP4057 Charger Module

To create the bluetooth integrated metal detector wand, a set of metal detector modules from Amazon(JESSINIE Metal Detector Module) was chosen for their cost effectiveness, rectangular geometry allowing them to be placed into a row in a functional wand design. The piezo buzzers on each module were desoldered and replaced with LEDs to provide a visual indication of where on the wand metal was being detected, and the signal line at the anode of the LED for each module was connected to the microcontroller through a voltage divider in order to read the detection signal using the ESP32's onboard analog to digital converter(ADC). For the microcontroller on the metal detector wand, an ESP32C3 Supermini development board was selected because it provides wireless functionality through Bluetooth Low Energy(BLE), ADC functionality, and I2C communication interfaces through its general purpose input and output(GPIO) pins. In addition to this, the Supermini's USB-C flashing capability and Arduino IDE compatibility supported convenient rapid prototyping. In order to power the wand with integrated functionality for reading battery status and convenient USB-C rechargeability, a 250 mAh lithium polymer(LiPo) battery was chosen along with a MAX17043 Fuel Gauge module which features the capability to read the charge of the battery as a state of charge(SoC) percentage, and communicate that information to the ESP32 over an I2C interface. To deliver rechargeability, a TP4057 module was selected because it features the capability to charge single cell LiPo batteries like the one selected through a USB-C interface delivering a convenient method of recharging.

Functional Specifications

1. mmWave Radar Imaging

1.1. Path generation logic

1.1.1. Inherited software will be implemented to generate optimized raster scan path for efficient scanning time

1.2. 2D Heatmap image reconstruction

1.2.1. Inherited software will be implemented to reconstruct 2D heatmap image from raster scan radar data

1.3. Gantry Structural Reinforcement

1.3.1. Gantry will be mechanically stabilized to reduce vibration and improve image quality relative to scanning speed

1.4. Motor Replacement

1.4.1. Gantry control stepper motors will be replaced with encoder stepper motors to improve positional accuracy

1.4.2. Positional tracking software will be updated to utilize positional data for improved image quality

2. RGB Imaging

2.1. Subject positional recognition

2.1.1. Inherited software will be implemented to identify subject position for path generation

2.2. Facial Image capture

2.2.1. Software will be updated to capture still image of subject facial features

3. Infrared Imaging

3.1. Infrared image capture

3.1.1. Software will be developed to interface infrared camera with raspberry pi and capture still infrared images

4. Metal Detection

4.1. Wireless Metal Detection Wand

4.1.1. A wireless metal detector wand will be developed using metal detector modules interfaced with and ESP32C3 Supermini development board over ADC pins to read metal detection signal

4.1.2. Metal detector wand will be capable of detecting metal objects >500g up to 5cm from sensor area with >75% testing accuracy

5. Object Recognition

5.1. mmWave radar heatmap knife detection model

5.1.1. An object detection model shall be trained to detect knives concealed under clothing on the test mannequin in the still mmWave radar heatmap images captured by the system

5.1.2. The object detection model shall achieve >60% true positive detection accuracy and >60% true negative detection accuracy on a test dataset of at least 5 positive and 5 negative test cases.

5.2. Infrared heatmap knife detection model

5.2.1. An object detection model shall be trained to detect knives concealed under clothing on the test mannequin in the still infrared heatmap images captured by the system

- 5.2.2. The object detection model shall achieve >60% true positive detection accuracy and >60% true negative detection accuracy on a test dataset of at least 5 positive and 5 negative test cases.
- 5.2.3. Test cases for the infrared imaging detection model will be collected using a knife secured against the body of the test mannequin. To simulate the thermal contrast expected between a human body and concealed knife, the knife will be cooled to be significantly cooler than room temperature for the capture of these test images
- 5.3. Visible light knife detection model
 - 5.3.1. An object detection model shall be trained to detect knives carried openly in the still visible light images captured by the system
 - 5.3.2. The object detection model shall achieve >60% true positive detection accuracy and >60% true negative detection accuracy on a test dataset of at least 5 positive and 5 negative test cases captured by placing a knife clearly within view of the system's visible light camera
- 6. Facial Recognition**
 - 6.1. Wanted persons facial detection and recognition system
 - 6.1.1. Facial detection and recognition libraries will be used to develop a custom python implementation of a wanted persons detection subsystem
 - 6.1.2. Vector embeddings will be precomputed and saved for a wanted persons database of at least 3 faces
 - 6.1.3. Still visible light images of at least 6 people, including at least 3 of the wanted persons will be captured using the system's visible light camera
 - 6.1.4. Each still image will be fed to the facial recognition workflow to identify the region of the image containing the subject's face, then vector embeddings will be calculated for that individual's facial features
 - 6.1.5. The vector embeddings of each test subject's face will be compared against those of each wanted person to determine whether there is a match
 - 6.1.6. Facial recognition workflow shall achieve >75% true positive detection accuracy and >75% true negative detection accuracy for each facial comparison case in the test dataset
- 7. Systems Integration**
 - 7.1. mmWave radar data transmission
 - 7.1.1. A control interface shall be able to trigger the mmWave heatmap capture procedure from the primary processing laptop
 - 7.1.2. The primary processing laptop shall receive mmWave radar data over wired interface with the mmWave evaluation board
 - 7.2. Visible light image data transmission
 - 7.2.1. A control interface shall be able to trigger the capture of still visible light images by the visible light camera connected to the raspberry pi 5 by the primary processing laptop
 - 7.2.2. The primary processing laptop shall receive visible light images captured by the raspberry pi over a wireless interface
 - 7.3. Visible light image data transmission
 - 7.3.1. A control interface shall be able to trigger the capture of still infrared images by the infrared camera connected to the raspberry pi 5 by the primary processing laptop

- 7.3.2. The primary processing laptop shall receive infrared images captured by the raspberry pi over a wireless interface
- 7.4. Metal detection data transmission
- 7.4.1. The esp32 based metal detection wand shall be able to transmit time series boolean data indicating the presence or absence of detected metal over bluetooth to the processing laptop

8. User Interface

8.1. Control panel dashboard graphical user interface

- 8.1.1. A custom python graphical user interface shall be developed to trigger data capture and AI inference workflows and surface relevant data and insights to the user on the primary processing laptop
- 8.1.2. Control panel shall feature a button to initiate the mmWave radar heatmap capture procedure, mmWave object recognition workflow shall initiate automatically following the completion of the scan and weapon detection insights shall be surfaced on the dashboard
- 8.1.3. Control panel shall feature a button to initiate infrared still image capture, infrared object recognition workflow shall initiate automatically and surface insights on the dashboard
- 8.1.4. Control panel shall feature a button to initiate visible light still image capture, facial recognition workflow shall initiate automatically and surface insights on the dashboard

System Design Overview

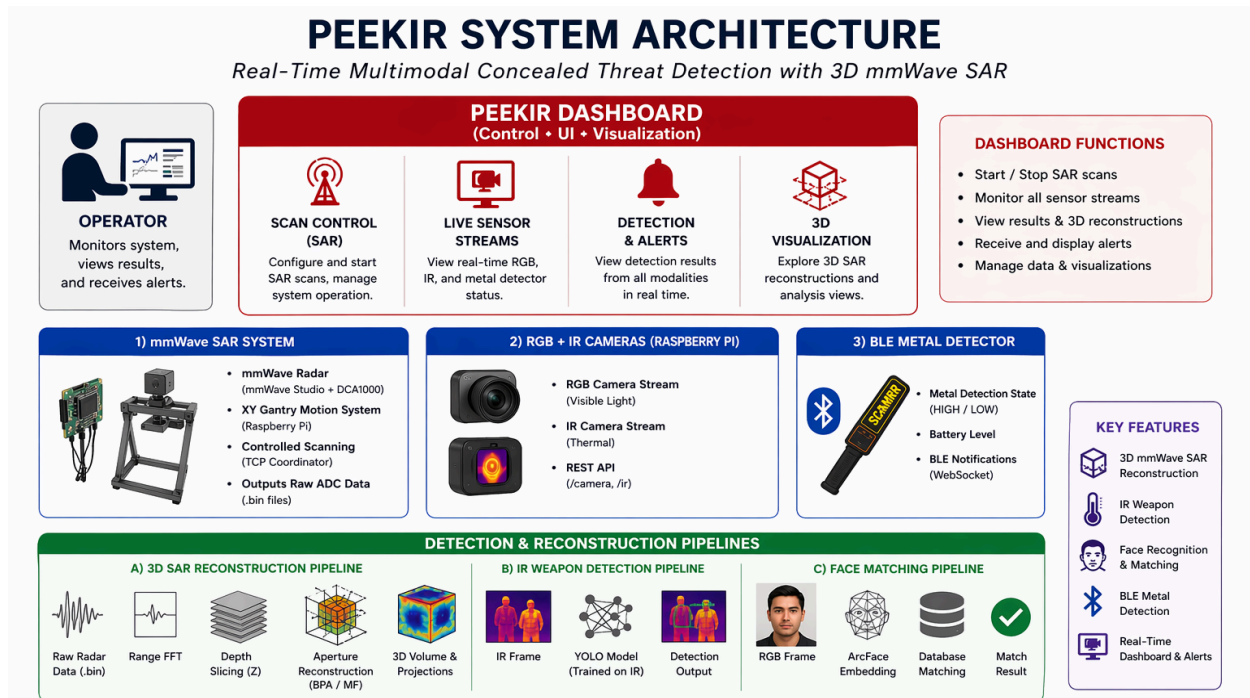


Figure 14: PeekIR System Architecture

Testing and Validation Approach

A systematic testing and validation procedure was carried out across all primary sensing modalities, including millimeter-wave (mmWave) radar imaging, facial recognition, infrared (IR) weapon detection, and metal detection, in order to assess the PeekIR system's performance and dependability. To evaluate detection accuracy, false positive and false negative rates, and overall system behavior, each subsystem was evaluated separately and in combination using both positive and negative situations.

Predefined test cases that mimic real-world security scenarios were used to conduct testing in controlled environments. Verifying that each sense modality operates as anticipated and evaluating the efficacy of the multimodal detection strategy were the goals of validation.

Facial recognition

A total of forty test cases were used to assess the face recognition module: ten negative instances (individuals who were not registered) and thirty positive cases (individuals who were expected to be identified or highlighted). The algorithm identified 24 out of 30 people as real positives with an accuracy of 80% for positive test cases. Six false negatives occurred, meaning that those who ought to have been flagged were not identified by the system. There were no instances of misclassifications.

The algorithm properly identified all ten people as genuine negatives with zero false positives in negative test situations, achieving 100% accuracy. Strong accuracy is demonstrated by these results, but they also emphasize the need for increased resilience in difficult situations like occlusion or change in face characteristics.

Infrared (IR) Weapon Detection Testing

A total of 100 cases—50 positive (weapon present) and 50 negative (no weapon present)—were used to test the IR weapon detecting module. With three false negatives, the system's accuracy for positive test cases was 94%, successfully identifying weapons in 47 out of 50 cases. The system produced zero false positives and 100% accuracy for negative test situations. Strong detection capabilities and dependability are demonstrated by these data, especially when it comes to preventing false alarms.

Metal Detection Testing

The capacity of the metal detecting subsystem to detect the presence of metallic items linked to possible hazards was assessed. Both control situations with no metallic objects and scenarios with metallic things on a patient were tested.

High sensitivity to conductive materials was demonstrated by the system's effective detection of metallic items in all tested positive circumstances. The system reliably distinguished between metallic and non-metallic situations, as evidenced by the absence of false positives in negative test instances.

Testing, however, also revealed that the metal detector is non-selective, which means that it finds any metallic object, regardless of how dangerous it is. This emphasizes the necessity of integrating with additional modalities (IR and mmWave) in order to differentiate potentially dangerous objects (such phones and keys) from harmless ones.

mmWave Radar Testing

The mmWave radar subsystem's capacity to produce spatial imaging data for the identification of hidden objects was assessed. Owing to technology limitations, mmWave testing prioritized manual inspection and visualization over automatic detection. The mmWave system can provide depth-resolved scans within an optimal range of around 200 mm to 400 mm, with a scan period of about 30 minutes for a 150 mm × 150 mm area, according to testing.

Since automatic categorization was not used in this iteration, it was necessary to manually analyze the resulting radar pictures in order to identify anomalies consistent with hidden items. These findings demonstrate the speed and automation constraints of mmWave sensing, but they also prove its viability as a supplementary modality.

Validation Summary

The testing and validation process confirms that the PeekIR system is capable of:

- Detecting concealed weapons using IR sensing with high accuracy
- Identifying metallic objects reliably through metal detection
- Generating spatial imaging using mmWave radar for concealed object analysis
- Performing facial recognition with strong precision and no false positives
- Leveraging a multimodal approach to improve overall detection confidence

While the results demonstrate promising performance, several limitations must be acknowledged:

- The system is an exploratory proof of concept and not production-ready
- Testing was conducted in controlled environments and may not fully reflect real-world conditions
- The mmWave subsystem has long scan times and requires manual interpretation
- The metal detector lacks object classification capability
- False negatives remain a concern in facial recognition and weapon detection
- The system relies on human operators, with machine learning intended to assist rather than replace personnel

Test Objectives

- Verify that the gantry system is mechanically stable, properly homed, and capable of accurate and repeatable motion within defined physical boundaries
(*TC-01, TC-04, TC-05, TC-06, TC-07, TC-08, TC-09*)
- Verify that optimized scan paths are correctly generated based on camera detection results and executed by the gantry system
(*TC-03, TC-05*)
- Verify that the RGB camera subsystem is capable of detecting a subject and performing facial recognition with acceptable confidence and latency
(*TC-02, TC-25, TC-26, TC-27, TC-28, TC-29, TC-30*)
- Verify that the mmWave radar subsystem successfully collects data and produces readable and accurate heatmap outputs
(*TC-10, TC-11, TC-12, TC-13*)
- Verify that the infrared (IR) sensor subsystem captures thermal data correctly and consistently across the defined field of view
(*TC-19, TC-20, TC-21, TC-22, TC-23, TC-24*)
- Verify that machine learning models correctly process sensor data, generate valid classifications, and output confidence scores within required processing time limits
(*TC-15, TC-16, TC-17, TC-18*)
- Verify that the Bluetooth-enabled metal detection wand reliably detects metal objects and transmits detection events to the computing device
(*TC-31, TC-32, TC-33, TC-34, TC-35, TC-36, TC-37*)

- *Verify that encoder-equipped motors provide accurate real-time position feedback and that positional data is correctly synchronized with IR and mmWave sensor data (TC-38, TC-39, TC-40, TC-41)*
- *Verify that all subsystems operate correctly when integrated and that the complete system meets defined functional, accuracy, and timing requirements (TC-01 through TC-41)*

Test Approach Phases

Incremental Testing -

Before system integration, incremental testing is done to confirm that each subsystem is operating correctly. Before communicating with other components, each subsystem is individually tested to ensure that it satisfies its functional requirements. Verification of gantry motion and homing, RGB camera detection and facial identification, mmWave radar data collection, infrared sensor heat capture, machine learning inference, and metal detecting wand capability are all examples of incremental testing. Faults can be found and fixed at the subsystem level during this phase.

Integration Testing -

Integration testing will show that the subsystems function properly when coupled. In order to guarantee compatibility and synchronization, data flow and control signals between components are verified during this stage. Verification of camera-based detection driving scan path generation, gantry execution of scan paths, sensor data acquisition during gantry motion, Bluetooth transmission of detection signals, and end-to-end data processing through the machine learning pipeline are all included in integration testing. Finding interface or timing problems across subsystems is the aim of this phase.

System Testing -

System testing checks how effectively the whole Hostile Weapon Detection system works together as a complete unit. This phase checks that all parts of the system work together to meet the requirements for function, accuracy, and timing. System testing checks that the system works reliably under certain test conditions. It also measures how long it takes to process information and runs a complete test from detecting the subject to identifying the weapon.

Test Items

- Gantry mechanical structure, ball screws, encoder-equipped motors, independent motor drivers, and encoder feedback processing logic
- Gantry homing sensors, boundary enforcement, and motion control logic
- Optimized scan path generation algorithms
- RGB camera subsystem for subject detection and facial recognition
- mmWave radar subsystem for data collection and heatmap generation
- Infrared (IR) sensor subsystem for thermal data capture
- Machine learning models for classification and confidence scoring
- Bluetooth-enabled metal detection wand
- Bluetooth communication and data reception on the computing device
- System-level coordination and integration of all subsystems

Features in Scope

- Accurate gantry positioning and repeatable motion execution
- Camera-based subject detection and facial recognition
- Generation and execution of optimized scan paths
- mmWave radar scanning and heatmap output generation
- IR thermal image capture and consistency verification
- Machine learning inference, confidence scoring, and processing duration
- Metal detection using the handheld wand
- Reliable Bluetooth communication between the wand and computing device
- End-to-end system operation from detection through classification

Features out of Scope

- Operation in low-light or outdoor environments
- Firearm detection
- Detection of multiple subjects simultaneously
- Face concealment or identity spoofing detection
- Full-body scanning beyond the defined scan area
- Long-term reliability or stress testing

Test Plan

Demo #	Test Case #	Description	Inputs/Precondition	Test Steps	Expected Results	Pass/Fail Criteria
Demo 1 - Gantry Optimization Verification of Functionality	TC-01	Ball screw installation inspection	Install ball screw onto existing gantry system	Visually and manually inspect the stability and rigidness of the installed ball screws and mounts on the gantry	Ball screw gives little tolerance when the gantry is shaken or while operating	Pass if ball screw installation is physically secure and stable. Fail if ball screw is not compatible or stable
	TC-02	Object detection verification	Place mannequin in front of system at a fixed distance while the camera is powered	Execute detection program	Mannequin upper torso is identified as human	Pass if detection is above 0.7 (70%), fail if otherwise
	TC-03	Path generation algorithm verification	Position gantry in front of mannequin at a fixed distance	Execute path generation program	Vertical snake path is created inside the mannequin boundary	Pass if path is generated and matches with the boundary, fail if otherwise
	TC-04	Gantry motion test	Gantry powered on with path loaded	Execute gantry motion program	Gantry follows the optimized path	Pass if gantry follows path, fail if otherwise
	TC-05	Gantry accuracy test	Gantry path loaded and laser temporarily installed on carriage	Run gantry carriage over mannequin	Carriage does not deviate by more than 8 inches	Pass if deviation < 8 inches, fail if otherwise

	TC-06	Physical boundary test	Gantry powered on with path loaded	Run gantry carriage over mannequin	Carriage does not exceed physical boundary	Pass if gantry doesn't exceed its physical limits, fail if otherwise
	TC-07	Hall effect homing sensor test	Install homing sensors on the gantry system	Manually hover a magnet over the hall effect sensor and hover the gantry carriage over the sensor	Hall effect sensor LED illuminates when a magnet is near and does not illuminate when a magnet is not there	Pass if homing sensor LED illuminates when a magnet is less than 5mm away, fail if otherwise
	TC-08	Duration test	Gantry powered on with path loaded	Complete the gantry path	Duration less than 10 minutes	Pass if duration < 10 minutes, fail if otherwise
	TC-09	Return to origin completion test	Gantry powered on with path loaded	Complete the gantry path	Carriage returns to origin	Pass if carriage reaches origin, fail if otherwise
Demo 2 - verification of mmWave functionality	TC-10	Flashing firmware (initiation)	Correct firmware flashed on the board	Flash device with the latest firmware, reboot device and application, and query system info	Device boots with latest firmware	Pass if flash is successful, fail if errors occur
	TC-11	CSV output generation	Scan completed and output is writeable	Export the output, verify directory location, and open CSV	CSV is produced correctly	Pass if file exists and is viewable, fail if errors occur
	TC-12	Heatmap generation	Processed CSV and plotter program is installed	Execute heatmap script, export heatmap, and view the heatmap	Heatmap is saved and output is viewable	Pass if image file exists and can be viewed, fail if otherwise
	TC-13	RMIST-Net hybrid algorithm test	Dataset loaded and program is configured	Run algorithm on the collected data and heatmap	Correctly predicts and smooths heatmap resolution	Pass if threshold is acceptable (>70% accuracy), fail if otherwise
	TC-14	Program duration test	Dataset labeled and script is configured and executed	Collected data is in progress for processing	Time of program processing is less than 5 minutes	Pass if duration of processing is < 5 minutes, fail if otherwise

Demo 3 - Machine learning confidence scoring	TC-15	Heatmap output test	Raw dataset is converted to heatmap and smoothed with algorithm	Execute script to generate heatmap and save the output	Heatmap image produced at correct resolution and saved to the directory	Pass if file exists and opens without error, fail if otherwise
	TC-16	Model input test	Model is trained, deployed, and accepts heatmap as input	Load heatmap and feed into the model as an input	Model accepts heatmap without error	Pass if no error occurs and outputs is as expected, fail if otherwise
	TC-17	Model confidence test	Model loaded with trained weight and test dataset is available	Load model with the heatmap and record confidence score	Model outputs classification of object in heatmap and tags confidence score	Pass if model accurately labels the object and tags a confidence score > 0.7, fail if otherwise
	TC-18	Model processing duration test	Heatmap and trained model is available	Load the model and record at start of processing and record at end of processing	Processing time measured is less than 5 minutes	Pass if model classification processing time is < 5 minutes, fail if otherwise
Demo 4 - Verification of IR Sensor Functionality	TC-19	IR sensor power-on verification	IR sensor connected to power and processing unit	Power on the system and initialize the IR sensor	IR sensor powers on and initializes without error	Pass if sensor initializes successfully, fail if sensor does not power on or throws errors
	TC-20	IR image capture test	IR sensor powered and configured	Execute IR capture program and save output	Thermal image is captured and saved to directory	Pass if image file exists and is viewable, fail if otherwise
	TC-21	Thermal response test	Heat source (e.g., human hand or warm object) present	Place heat source in front of IR sensor and capture image	Heat source appears with higher intensity than background	Pass if heat source is distinguishable from background, fail if otherwise
	TC-22	IR field-of-view verification	IR sensor mounted on gantry	Place heat source at multiple positions within expected FOV	Heat source appears in correct relative positions in image	Pass if object appears within expected FOV bounds, fail if otherwise

	TC-23	IR sensor data consistency test	Stable ambient environment	Capture multiple IR images over time	Thermal readings remain consistent across captures	Pass if variation is within acceptable range, fail if otherwise
	TC-24	IR sensor integration test	IR sensor and processing pipeline configured	Run system with IR sensor active during gantry operation	IR data is captured and processed without interrupting system	Pass if system runs without errors, fail if otherwise
Demo 5 – Verification of Facial Recognition (RGB Camera) Functionality	TC-25	RGB camera stream initialization test	RGB camera connected and powered; facial recognition program installed	Start camera stream and launch recognition program	Live video feed displays without errors	Pass if video feed loads with no errors, fail if stream fails or crashes
	TC-26	Face detection test	Subject in view with adequate lighting	Stand in front of camera; run face detection	Face bounding box appears on subject's face	Pass if face is detected within 2 seconds, fail if no detection
	TC-27	Enrollment / profile creation test	Subject identity label available; enrollment mode enabled	Enroll one subject and save profile	Profile is saved and can be loaded later	Pass if profile saves and reloads correctly, fail if missing/corrupt
	TC-28	Recognition (known subject) test	Enrolled subject available	Run recognition with enrolled subject in view	System labels subject as the enrolled identity	Pass if correct identity is displayed with confidence ≥ 0.7 , fail otherwise
	TC-29	Rejection (unknown subject) test	A second, non-enrolled subject available	Run recognition with unknown subject in view	System does not incorrectly label as known	Pass if labeled "unknown" / no-match, fail if misidentified as a known user
	TC-30	Recognition latency test	System running in normal mode	Measure time from face entering frame to identity output	Recognition completes within target time	Pass if latency < 2 seconds, fail otherwise
Demo 6 – Verification of Metal Wand Functionality	TC-31	Power-on and self-check test	Wand assembled; battery charged	Power on wand	Wand powers on and enters ready state	Pass if wand powers on reliably, fail if it does not power on

	TC-32	Metal detection functional test	Test objects: metal + non-metal	Bring metal near sensor; repeat with non-metal	Detects metal; no trigger on non-metal	Pass if metal triggers and non-metal does not, fail otherwise
	TC-33	Detection threshold / distance test	Ruler/tape; standard metal object	Measure max distance where detection triggers	Threshold distance meets design target	Pass if detection occurs at/above target distance, fail otherwise
	TC-34	Bluetooth pairing test	Computer with Bluetooth enabled; wand in pairing mode	Pair wand to computer	Successful pairing without error	Pass if paired and recognized by OS, fail otherwise
	TC-35	Bluetooth message transmission test	Paired connection established; receiver program running on computer	Trigger metal detection and log received signal	Computer receives correct signal/message	Pass if message received within 1 second and correctly parsed, fail otherwise
	TC-36	Connection stability test	Paired and receiver running	Maintain connection while idle + repeated detections	Connection remains stable and doesn't drop	Pass if no disconnects during test window, fail otherwise
	TC-37	Range test	Open area; measured distance	Walk wand away from computer until failure; record range	Bluetooth works up to expected range	Pass if range $\geq$ target distance, fail otherwise
Demo 7- Encoder Positioning & Synchronization Verification	TC-38	Encoder position accuracy test	Encoded motors installed, calibrated, and homed	Command gantry to move to known coordinate (X,Y); measure physical position with calipers; compare to encoder-reported position	Encoder-reported position matches measured physical position within tolerance	Pass if position error ≤ 2 mm; Fail otherwise
	TC-39	Encoder repeatability test	System homed and calibrated	Move gantry to same coordinate 10 times; record encoder position each time; calculate	Encoder readings remain consistent across repeated trials	Pass if positional variance ≤ 1 mm; Fail otherwise

				variance		
	TC-40	Sensor-position synchronization test	IR and mmWave sensors active during scan; timestamp logging enabled	Move gantry in fixed increments; capture sensor data at each step; compare encoder timestamps with sensor timestamps	Sensor data aligns correctly with recorded gantry position	Pass if no observable spatial misalignment in reconstructed image; Fail otherwise
	TC-41	Image reconstruction spatial accuracy test	Test object placed at known ground-truth position	Perform full scan; generate mmWave heatmap and IR image; compare detected object position to known ground truth	Object appears in correct spatial location in generated images	Pass if positional error $\leq 5\%$ of scan range; Fail otherwise

Electronic/Hardware Design

Gantry Hardware System

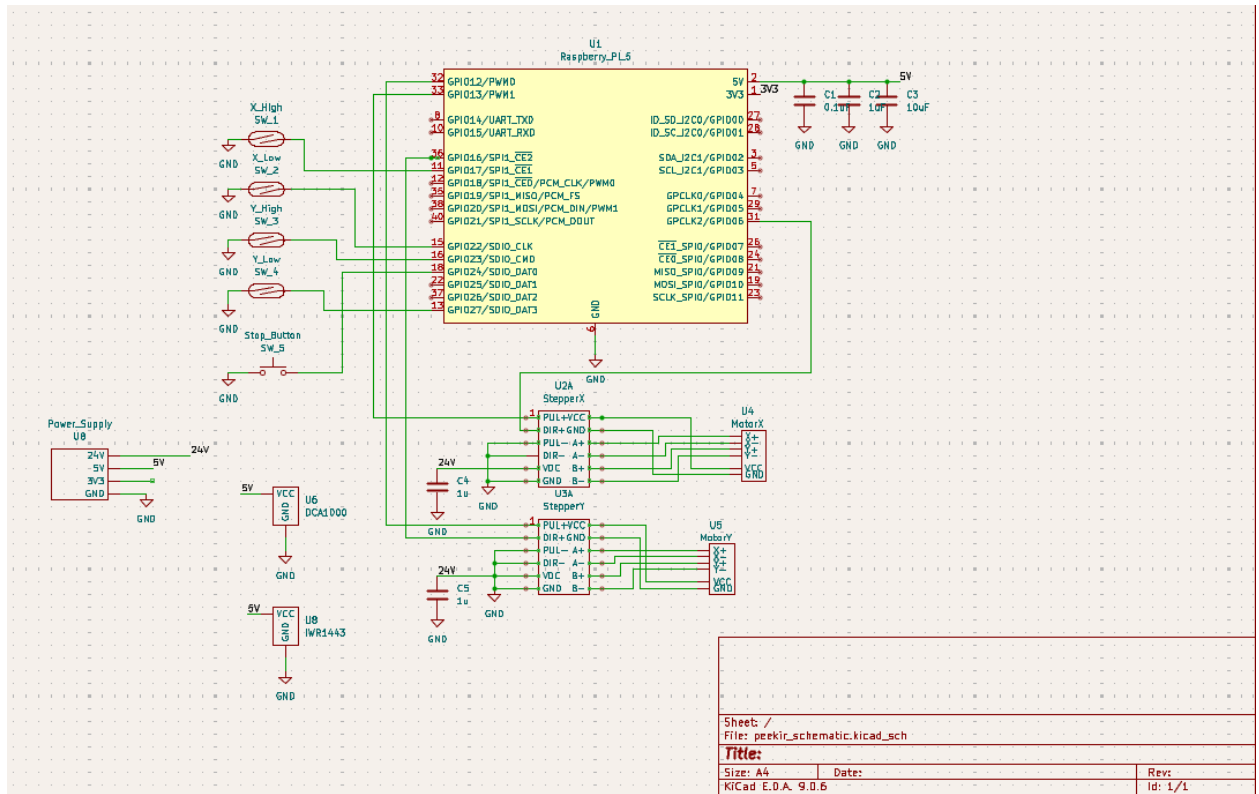


Figure 15: Gantry Hardware System Schematic

The gantry electronic system features the Raspberry Pi 5 as the central control node, which receives inputs from the four reed limit switches at the end of each axis of the gantry, used to calibrate position and prevent moving outside of the available, which would risk damaging components. The Pi also interfaces directly with the stepper driver, providing pulse width modulated (PWM) signals to drive motor steps. The schematic depicts the motor drivers interfaced with the stepper motors, which are powered by 24V from the power supply and take step pulses from the Pi for movement control. The reed switches rely on the 5V rail supplied by the Pi. The schematic also depicts the IWR1443BOOST and DCA1000EVM evaluation boards

which are mounted on the gantry and each powered through their own 5V barrel jack. The evaluation boards interface with the laptop over USB and ethernet connections.

Metal Detector Wand

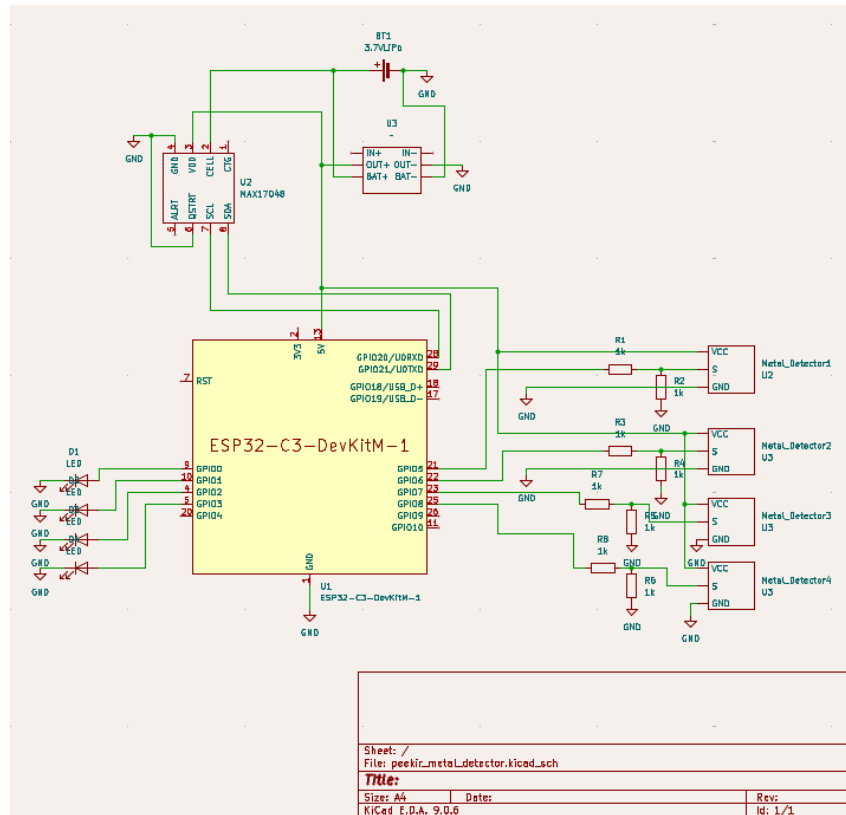


Figure 16: Metal Detector Wand Schematic

The central node of the metal detector schematic is the ESP32C3 Supermini development board depicted in the schematic above. The schematic depicts the four metal detector modules powered through VCC and GND, each with their signal line connected to a GPIO pin on the ESP32 through a voltage divider reducing the voltage from up to 5V to up to 2.5V, putting it into the range to be read by the ESP32 ADC. The schematic also depicts the four indicator LEDs, each corresponding to one of the metal detector modules. Above the ESP32, the wand power system is depicted including the battery, fuel gauge, and charger module. The charger module interfaces directly with the positive and negative ports of the battery, and provides output voltage to the rest

of the system. The Fuel Gauge module also interfaces directly with the battery through the CELL pin, and is powered through VDD from the charger output pin. The fuel gauge communicated battery status as a percentage over I2C using the Serial Data Line(SDA) and Serial Clock Line(SCL) connected to GPIOs 20 and 20.

Software Designs

PeekIR is a multi modal security scanning device where it combines radar imaging, infrared camera, RGB camera, metal detector and facial recognition analysis. This system is split between 3 different systems which are the Raspberry Pi 5, PC and an ESP32 (for metal detection) all these different systems are combined on the PC at the UI application where all the results and images are shown.

System Architecture Overview:

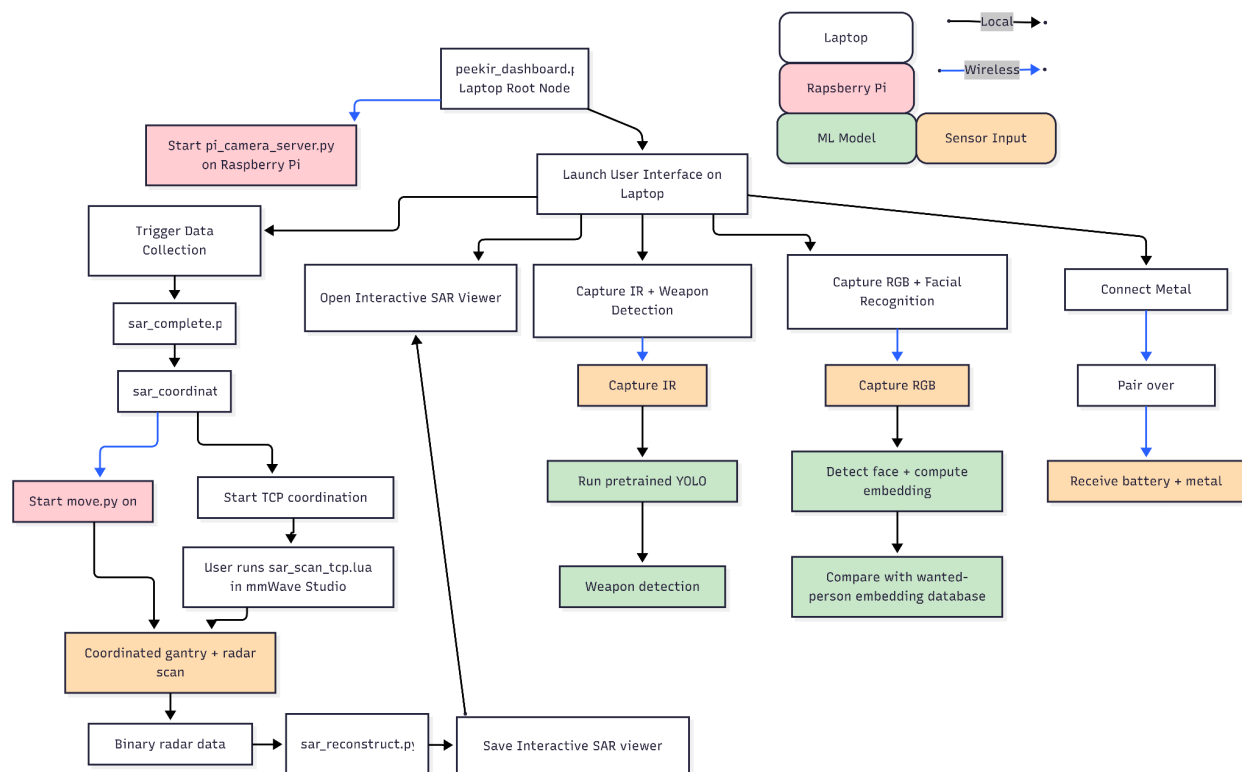


Figure 17: Software Architecture and Flow

Full Code repository can be found at: https://github.com/seanconway/PeekIR_Final

Through this system, these are some of the core functionalities PeekIR strives to deliver:

- Weapon detection in both infrared and radar images using the YOLOv8 model
- Detection of whether a person is present in the frame or not using object detection models
- Facial recognition capabilities to match a person to a criminal database using DeepFace
- Real-time camera streaming to constantly send images to be matched with the database
- Display of all the outputs in a web-based user interface
- Radar captures and operation from local software

1. Backend

- a. FASTAPI is used as the primary API
- b. Handles facial detection, image streaming, and camera integration
- c. Handles radar configuration and operation through API calls to the ar1 module

2. Frontend

- a. A user interface was created using a React-based application
- b. Front-end application is connected to the backend code through REST API endpoints

3. Machine Learning

- a. The YOLO model was used for weapon detection
- b. DeepFace was used for facial recognition and criminal database matching

4. Hardware Integration

- a. Raspberry PI was used for capturing facial images
- b. The PC offload server takes the image from the PI and sends it to the user interface to display the results
- c. Software configures, triggers, and saves data from the IWR radar and the DCA capture card

Data Acquisition & Processing

The software system takes four independent data inputs simultaneously. The inputs are as follows: mmWave radar, IR camera, RGB camera, and an ESP32 metal detector.

Key Files

- peekir_dashboard.py
- sar_coordinator.py
- sar_reconstruct.py
- move.py
- gunDetect.py
- personDetect.py
- recognize_poi.py

peekir_dashboard.py

- Launches browser based graphic user interface (GUI)
- Remotely activates pi_camera_server.py via SSH automated by powershell script

sar_coordinator.py

- Creates a TCP server on port 5555. Expects two clients to connect to server:
 - Radar: lua script running on PC through mmWave Studio
 - Gantry: move.py on the PI
- For each row in the scan grid, the coordinator sends the gantry a move command (X position in mm), then waits for MOVE_COMPLETE, then sends the radar an ARM -> Capture sequence.
- DCA1000 writes bin file of raw I/Q samples for that row
- Process repeats row-by-row until full aperture is covered.

sar_reconstruct.py

- Reads each bin file which corresponds to a row in the scanning grid, the parses the int16 I and Q samples across the RX antenna channels into a 3d NumPy array
- 2D FFT is applied along the fast-time (range) dimension for range-Doppler processing
- Reconstruction algorithm is applied and the result is projected onto XY, XZ, and YZ planes, then saved as an interactive Plotly HTML viewer.

gunDetect.py

- Takes image input from IR stream and passes it to the YOLO model.
- Each detection returns a dataclass containing class_id, label, confidence, as well as bounding box coordinates.
- Script triggered by live IR frame endpoint

personDetect.py

- YOLO pretrained model take input of RGB image, and returns the person detection output
- Takes in the RGB feed
- First runs the code and checks if it finds a match in the database. If it is false, then it runs another verification run with a slightly lower confidence threshold and larger image size to double check with the accuracy.

recognize_poi.py

- DeepFace with ArcFace backbone used along with pre-built database
- Scans the database, computes ArcFace embedding for each face image
- At query, this script loads the embeddings JSON and calls deepface to compute the distance between suspect image and every POI embedding
- Returns closest match

App.tsx

- Streams the IR feed live and shows if there is a weapon detected in the next frame

- Polls the IR api to trigger the weapon detection in real time and draw a bounding box is a weapon is detected.
- Streams live RGB feed and shows if there is a POI image in the next frame
 - Also displays the criminal background underneath the RGB images
- The user can capture snapshots of IR and RGB feeds as a still image is needed
- Listens for the weapon detection signals and keeps it displayed on the user interface

To run a complete scan:

Windows terminal

```
python sar_coordinator.py -W 280 -H 40 -r 40 -o "C:\scans\output"
```

Raspberry Pi terminal

```
python3 move.py tcp host=<WINDOWS_IP> port=5555
```

Windows

```
python sar_reconstruct.py --folder "C:\scans\output\scan_0" --zindex 300mm --plotly
```

Infrared & RGB camera (pi_camera_server.py):

First a FastAPI is run on the PI to capture both the IR and RGB feeds.

- RGB feed used the picamera2 to capture the pi camera frames
- IR uses V4L2 to capture the frames from the IR camera which is connected to the PI

Both these feeds encode their respective captures as JPEG and store them in a cache that is easily accessible to the PC. The backend on the PC then accesses these caches and sends them to the frontend as MJPEG via HTTP.

Pi camera service endpoints:

GET /api/camera/frame — RGB JPEG

GET /api/camera/stream — RGB MJPEG stream

GET /api/ir/frame — IR JPEG

GET /api/ir/stream — IR MJPEG stream

Metal Detector (metal_detector.py)

An ESP32 was used for the metal detection and the results were sent over BLE.

metal_detector.py is run on the PC where the detector asynchronously sends the metal detection results and the battery status to the PC, which is then displayed on the user interface.

- PIN HIGH/ PIN LOW - metal detected/ no metal detected
- BAT: % - battery percentage

The PC backend uses the GET/api/metal-status api. The frontend occasionally uses this and displays the results on the interface. If there is a threat, then an audio warning is triggered.

Mechanical Designs

Mechanical System Overview

The mechanical subsystem was designed to provide a stable, repeatable, and technically practical platform for the prototype system's sensing hardware. Its main purpose was to support the mmWave radar scan process, maintain sensor alignment, and integrate the added infrared and metal-detection hardware into a usable physical system. Because SAR imaging depended on knowing the radar position accurately throughout the scan, the mechanical design directly affected image quality, scan consistency, and overall system performance.

The prototype system used a gantry-based architecture inherited from a previous capstone team and then improved by PeekIR. The major mechanical elements included the gantry base, ball screws, stepper motors, motor carriage, metal detector enclosure, and IR camera mount. The mechanical design needed to meet several system requirements; it had to provide rigid support for the radar hardware, allow controlled X-Y movement for scan collection, reduce vibration during operation, and package the added sensing hardware in a clean and repeatable way.

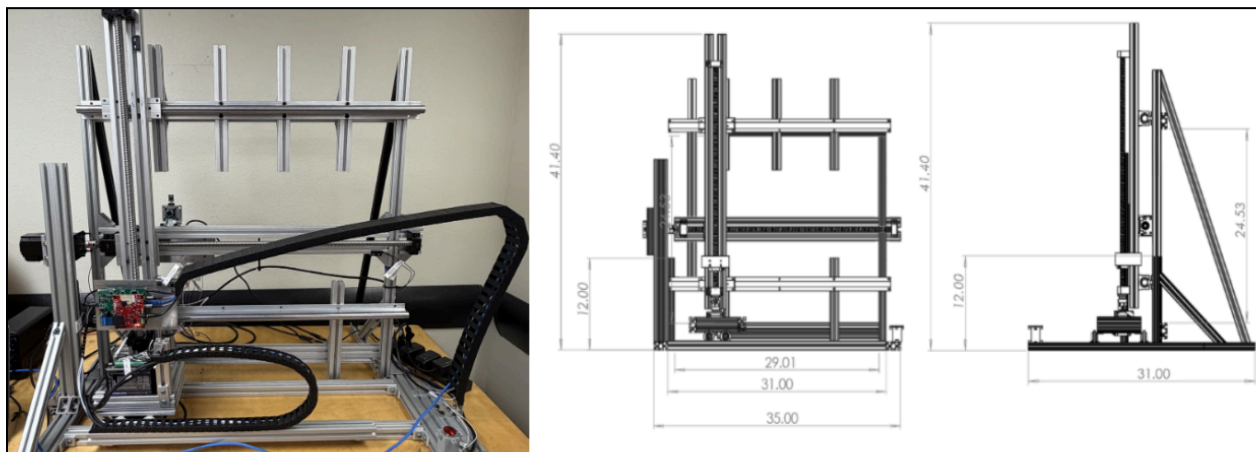


Figure 18: Overall Gantry vs. Full Mechanical System Drawing

Motion and Positional System

The motion subsystem moved the mmWave radar module across a defined scan region so radar returns could be collected from multiple known positions. This motion was essential to SAR reconstruction. The radar image was built from data gathered across a physical aperture, so the system depended on consistent movement and reliable positioning.

To produce this motion, the gantry used two ball screws, one horizontal and one vertical, driven by two stepper motors. The ball screws converted rotary motor motion into controlled linear travel in the X and Y directions. Attached to the intersection of the ball screws was a frame that held the mmWave radar scanner, allowing it to move in both axes. This arrangement provided a mechanically rigid and repeatable method of positioning the radar module across the scan region.

A major improvement made by PeekIR was replacing the inherited motors and drivers with closed-loop motor hardware. The original TB6600 drivers and 23HS45-4204S motors were replaced by CL57T drivers and 23HS45-4304D motors to improve positional control. The goal of this upgrade was to improve motion consistency and reduce the likelihood of missed steps during scanning.

A secondary encoder-logging approach that could have allowed external recording of motor position and speed during gantry operation was also explored. The intended idea was for the CL57T drivers to continue using encoder feedback internally for closed-loop correction while a separate external processor independently logged motion data such as position traveled, speed, and repeatability.

Because the encoder signals provided by the motor-driver system were differential quadrature pairs, labeled A+/A- and B+/B-, they were not directly compatible with a typical microcontroller GPIO input. For that reason, the team investigated converting the differential encoder signals into single-ended logic-level signals before counting them externally.

To do this, an RS-422 differential line receiver called “TI AM26LV32” was planned to be used. The AM26LV32 would have taken the differential encoder pairs and converted them into clean A/B square-wave outputs. These outputs would then have been counted by an ESP32-C3 microcontroller in real time to produce a running position counter and counts-per-second data for speed. The ESP32 was considered for this task because it could have handled time-sensitive pulse counting more reliably than a Raspberry Pi running a non-real-time operating system, especially at higher motor speeds. The ESP32 would then have communicated that data to the main Raspberry Pi over UART.

The physical wiring concept was also important. The encoders needed to remain connected to the CL57T drivers on both axes so the closed-loop control loops could continue functioning as designed. Because of this, the A/B encoder signals would have been tapped in parallel rather than removed from the driver. In that approach, the CL57T drivers would still have received encoder feedback for closed-loop correction while the external measurement system simultaneously observed the same differential signals for logging. The differential quadrature outputs from both drivers would then have been routed into the AM26LV32 receiver channels, and the receiver would have produced single-ended A/B outputs for each axis for the ESP32 to count.

The AM26LV32 would have been powered from the microcontroller’s 3.3 V pin and kept enabled so it could output clean A/B signals into the ESP32. This approach would have allowed the CL57T drivers to continue operating normally in closed-loop mode while also attempting to log position and speed for both motors without disturbing the control loop. Keeping the encoder signals differential up to the AM26LV32 would also have helped reduce noise in a high-current stepper system.

However, during testing, the external encoder readback proved inconsistent. Counts would reset, skip, or freeze, and the resulting position data could not be trusted. Because of that, external encoder logging was not ultimately implemented as part of the final system.

The motor carriage supported the radar hardware while translating it vertically along the gantry. Mechanically, the carriage had to remain aligned and stable while carrying the mmWave module through repeated scan motions. This was important because carriage deflection or vibration would create spatial error in the scan data.

Structural Design and Packaging

The structural design was based on an aluminum T-slot frame that served as the main gantry chassis. This approach was well suited to the project because it provided a rigid yet modular platform that could be modified as the design evolved. Since the prototype system was inherited in part from a previous team, modular framing made it easier to reinforce the structure and add new subsystems without redesigning the entire platform.

One of the major mechanical improvements to the existing system was reinforcing the gantry to reduce vibration. Structurally, this mattered because vibration reduced scan repeatability and lowered radar image quality. Additional support beams and machined aluminum support members were added to stiffen the frame and improve overall stability during operation, especially near the upper ends of the gantry, which showed greater vibration during test runs.

The packaging design also included custom integration for the new sensing subsystems. The metal detector was placed into a dedicated enclosure so the sensing modules could be aligned in a row and used as a practical handheld wand. This improved durability, usability, and subsystem organization compared to leaving the hardware exposed.

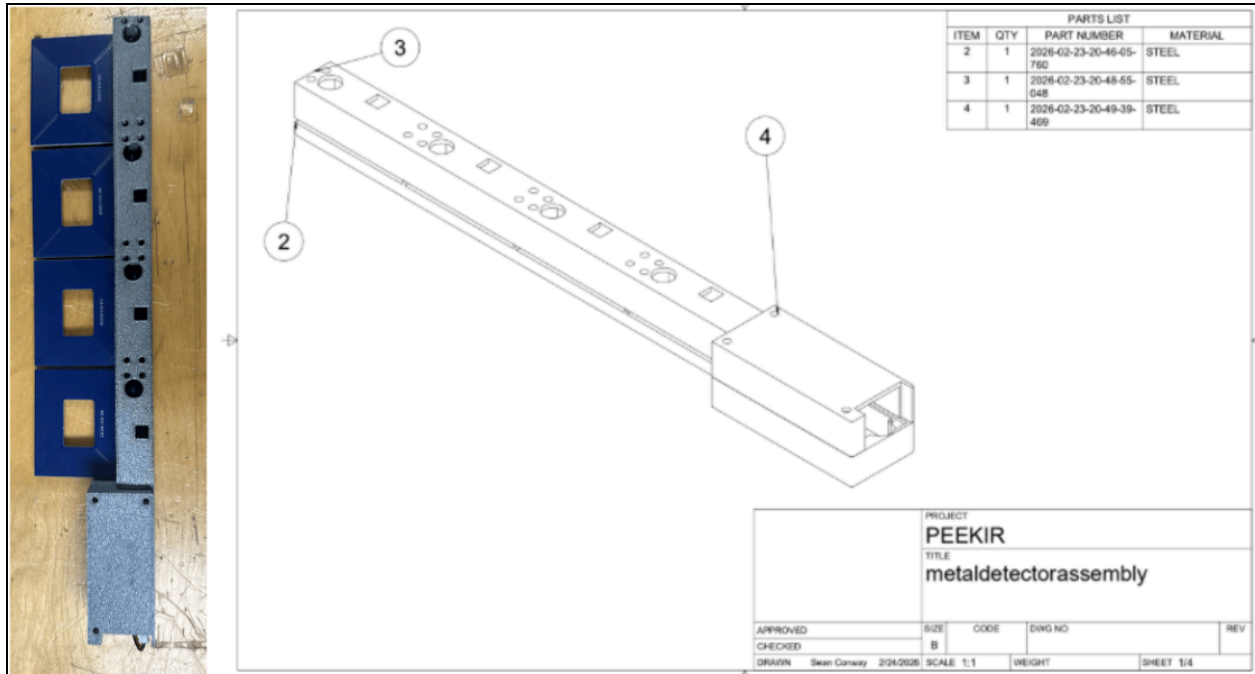


Figure 19: Actual Metal Detector Assembly vs. Metal Detector Drawing

The IR subsystem also required a custom camera mount. That mount was designed to hold the camera in a fixed orientation on the frame while also including a wire-routing groove so the cable could sit inside the part more cleanly. This improved alignment, protected the cable path, and made the thermal subsystem easier to integrate into the overall platform.

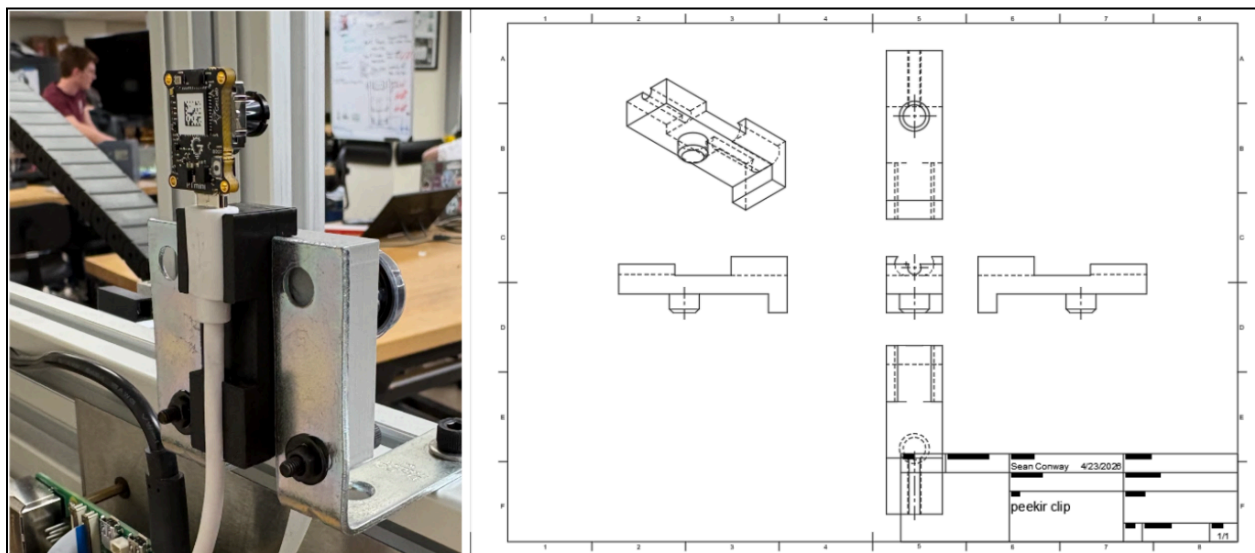


Figure 20: Actual IR Camera Mount vs. IR Camera Mount Drawing

Overall Theory of Operation for Mechanical System

The mechanical system operates by providing both a rigid structural frame and a controlled motion mechanism for the sensing hardware. The gantry base acts as the chassis and physical reference frame. The motors rotate the ball screws, and the ball screws convert that rotary motion into linear travel along the X and Y axes. This moves the motor carriage and mounted mmWave radar to successive positions within the scan region. As the carriage moves through the scan path, the radar captures data at known locations. Those positions are later used by the reconstruction pipeline to generate spatial radar images. The quality of the radar image depends on the structural stiffness of the frame, the consistency of carriage motion, and the repeatability of the scan path.

The structural members operate by resisting deflection and vibration during scan motion. The added support beams increase stiffness so that the carriage does not shift significantly from its intended position while moving. The metal detector enclosure supports and protects the handheld wand hardware, while the IR camera mount fixes the thermal camera in a stable position and manages cable routing. Together, these components allow the prototype system to function as one physically integrated platform.

Overall, the mechanical design is sound because it provides the structure and motion needed for repeatable radar scans. It addresses system needs by supporting the sensing hardware, improving positional consistency, reducing vibration, and packaging the added IR and metal-detection subsystems into usable forms. Each major element of the design has a clear mechanical role: the frame provides support, the ball screws and motors generate controlled motion, the carriage positions the radar, the reinforcements increase stiffness, and the custom mounts package the remaining camera hardware.

The mechanical subsystem was validated through gantry motion testing, structural observation during scan runs, and repeatability-focused evaluation of carriage movement. These tests confirmed that the reinforced frame and upgraded motion hardware provided improved stability and more consistent scan motion for the final prototype.

Project Status

Cost Analysis

The team is divided into various subteams for this project. There are three major subteams consisting of two people each. One team assigned to software development, another to focus on mechanical and hardware development, and lastly one team to focus on the radar imaging development. Although there were subteams for each subsystem, everyone helped each other out in all the tasks. The team members would actively assist one another when they had no work on their assigned tasks. All the tasks were tested with multiple testing rounds. This streamlined workflow is built in a way that there are a proportional number of members assigned to the core disciplines. At the same time, the team structure allows for a flexible shift in roles given various deadlines and requirements. This helped all the team members to always be aware of various tasks and deliverables at any given time. *Figure 21* is a projected table which shows the salary for every member on the team. The hourly costs for each role were determined and multiplied by the number of hours of work each of the members was assigned to complete tasks.

Table 1: Resource Cost Analysis Table

Member	Rate	Hours	Total
Rajat (PM/Hardware)	\$25/H	110	\$2,750
Sean (Systems Integration)	\$30/H	147.75	\$4,432.50
Havish (Integration/Test)	\$28/H	109	\$3,052
Rayaan (Software)	\$24/H	120.5	\$2,892
Vidhya (Software)	\$29/H	124.32	\$3,605.28
Trisha (Hardware)	\$32/H	117	\$3,744
Team Total	\$169/H	728.57	\$20,476

Bill of Materials

The budget assigned for this project by the sponsor was originally \$1000 and later reduced to \$750 at the start of the semester. **Table 1** shows the bill of materials of everything the team purchased this semester for the project. All the components listed were shortlisted after evaluating the cost effectiveness, ability to integrate with multiple components, functionality, and durability. This was established after thorough research and consultation with the sponsor of the project. The components which have an asterisk were purchased personally by the team members without using the budget provided. **Table 1.2** is the bill of materials from the previous team from where the gantry was inherited. Some of the components which are listed in the table were personal items of the previous team; hence the team had to repurchase them.

Item	Quantity	Unit Price	Total
New Items			
Power Strip	1.00	10.00	10
Power Supply	1.00	52.00	52
USB Extension Cables	2.00	10.00	20
Metal Detection Modules	5.00	3.00	15
FLIR LEPTON UWFOV Infrared Camera	1.00	143.00	143
Purethermal Mini USB	1.00	115.00	115
Rubber Feet	4.00	1.75	7
Support Beams	2.00	21.50	43
CL57T Stepper Drivers	2.00	36.39	72.78
23HS45 Stepper Motors	2.00	35.00	70
IR Breakout #2	1.00	115.00	115
Raspberry Pi 5	1.00	100.00	100
ESP32C3 Supermini	1.00	4.00	4
TP4057 module	1.00	1.80	1.8
MAX17043 Fuel Gauge Module	1.00	9.32	9.32
Raspberry Pi Pico	1	7.5	7.5
AM26LV32 Differential Line Receiver	1	1.8	1.8

Table 1: BOM of PEEKIR

Item	Quantity	Unit Price	Total
New Items			
Power Strip	1.00	10.00	10
Power Supply	1.00	52.00	52
USB Extension Cables	2.00	10.00	20
Metal Detection Modules	5.00	3.00	15
FLIR LEPTON UWFOV Infrared Camera	1.00	143.00	143
Purethermal Mini USB	1.00	115.00	115
Rubber Feet	4.00	1.75	7
Support Beams	2.00	21.50	43
CL57T Stepper Drivers	2.00	36.39	72.78
23HS45 Stepper Motors	2.00	35.00	70
IR Breakout #2	1.00	115.00	115
Raspberry Pi 5	1.00	100.00	100
ESP32C3 Supermini	1.00	4.00	4
TP4057 module	1.00	1.80	1.8
MAX17043 Fuel Gauge Module	1.00	9.32	9.32
Raspberry Pi Pico	1	7.5	7.5
AM26LV32 Differential Line Receiver	1	1.8	1.8
T-Slot Structural Bracket (4844N136)	8.00	8.27	66.16
Arducam Pi Camera 3 Wide	1.00	29.22	29.22
DCA1000EVM	1.00	720.00	720
Reed Switch	4.00	2.50	10
6ft CAT-6 Ethernet Cord	1.00	10.00	10
15-pin to 22-pin CSI Cable	1.00	7.97	7.97
Power Hub PCB	1.00	17.50	17.5
IWR1443 BOOST	1.00	358.81	358.81

Table 1.2: BOM of Previous Capstone team

Reproduction Cost

The reproduction cost is defined as the estimated cost required to build the project for the market on a large scale. As the project is a proof of concept and given a limited budget there was only so much the team could produce. If given a larger budget, the main reproduction costs would be primarily attributed to acquiring multiple higher quality radar sensors.

Product Evaluation

PeekIR was able to successfully meet all the requirements the team committed to deliver to the sponsor. The team was able to make significant improvements in mmWave radar image quality from TCP coordinator and postprocessing enhancements. Improvements in mmWave imaging time efficiency from using the snake path scanning were notable. On the mechanical side, improved positional estimation consistency improvements with internal encoder motor and driver replacement. The gantry structure was also reinforced by adding support beams to reduce vibration. Some improvements on the software side were the addition of facial recognition and criminal identification subsystem. Additionally the infrared imaging and weapon detection were very effective. Lastly, the addition of a Bluetooth-enabled metal detector as well. Overall demonstrating a fully integrated multimodal system through a unified user interface dashboard.